



Machine Learning Experiment Number 3

Name : Pranay Bhikaji Mestry Roll No : CS31

Aim : To implement a Multiple Linear Regression model from scratch using NumPy matrix operations to predict the selling price of used cars using multiple features from the CarDekho Used Car Dataset.

Learning Outcomes • Understand the working principle of Multiple Linear Regression.

- Perform data preprocessing and feature engineering.
- Convert categorical data into numerical features.
- Standardize numerical features.
- Implement the Ordinary Least Squares method using matrix operations.
- Train and test a regression model without using a ready-made regression function.
- Evaluate and interpret regression performance using RSS and R^2 .
- Understand the effect of individual features on the predicted price.

Dataset

Use the **CarDekho Used Car Dataset** (`car_data.csv`). [CarDekho Used Car Dataset](#)

Explain meaning of each attribute in this dataset in below comment section :

Here, **Selling_Price** is the target variable.

Software/Tools Required

- Python 3.x
 - Google Colab / Jupyter Notebook
 - NumPy
 - Matplotlib
-

Task 1: Set Up the Python Environment**

- Create a new Google Colab notebook.
 - Import **NumPy** for numerical and matrix operations.
 - Import **Matplotlib** for data visualization.
 - Do not use ready-made machine-learning regression functions such as `LinearRegression()` from Scikit-learn.
 - The regression model must be implemented using matrix operations.
-

```
In [18]: # TASK 1: SET UP THE PYTHON ENVIRONMENT

# Import NumPy for numerical and matrix operations
import numpy as np

# Import Matplotlib for data visualization
import matplotlib.pyplot as plt

# Display confirmation
print("Python environment set up successfully!")
print("NumPy version:", np.__version__)
```

Python environment set up successfully!
NumPy version: 2.0.2

Task 2: Load and Inspect the Dataset

- Download the `car_data.csv` file.
 - Upload the dataset to Google Colab.
 - Load the dataset into Python.
 - Display the first few records.
 - Determine the number of rows and columns.
 - Identify the numerical and categorical attributes.
 - Separate `Selling_Price` as the target variable.
-

```
In [20]: # TASK 2: LOAD AND INSPECT THE DATASET

import pandas as pd
from google.colab import files

# Upload CSV file manually
uploaded = files.upload()

# Get the uploaded file name
file_name = list(uploaded.keys())[0]

# Load the dataset
```

```

df = pd.read_csv(file_name)

# Display the first 5 records
print("First 5 records:")
display(df.head())

# Display number of rows and columns
print("\nNumber of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

# Display column names
print("\nColumn names:")
print(df.columns.tolist())

# Identify numerical attributes
numerical_attributes = df.select_dtypes(include=['int64', 'float64']).columns.

print("\nNumerical attributes:")
print(numerical_attributes)

# Identify categorical attributes
categorical_attributes = df.select_dtypes(include=['object']).columns.tolist()

print("\nCategorical attributes:")
print(categorical_attributes)

# Separate selling_price as target variable
y = df['selling_price']

print("\nTarget variable: selling_price")
display(y.head())

```

Upload widget is only available when the cell has been executed

Unlabeled widget is only available when the cell has been executed

in the current browser session. Please rerun this cell to enable.

Saving cardekho_dataset.csv to cardekho_dataset (5).csv

First 5 records:

	Unnamed: 0	car_name	brand	model	vehicle_age	km_driven	seller_type	ft
0	0	Maruti Alto	Maruti	Alto	9	120000	Individual	
1	1	Hyundai Grand	Hyundai	Grand	5	20000	Individual	
2	2	Hyundai i20	Hyundai	i20	11	60000	Individual	
3	3	Maruti Alto	Maruti	Alto	9	37000	Individual	
4	4	Ford Ecosport	Ford	Ecosport	6	30000	Dealer	

Number of rows: 15411
Number of columns: 14

Column names:
['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven', 'seller_type', 'fuel_type', 'transmission_type', 'mileage', 'engine', 'max_power', 'seats', 'selling_price']

Numerical attributes:
['Unnamed: 0', 'vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power', 'seats', 'selling_price']

Categorical attributes:
['car_name', 'brand', 'model', 'seller_type', 'fuel_type', 'transmission_type']

Target variable: selling_price

	selling_price
0	120000
1	550000
2	215000
3	226000
4	570000

dtype: int64

Task 3: Check and Handle Missing Data

- Check every attribute for missing or null values.
- Count the number of missing values in each column.
- If missing values are present, either remove incomplete records or perform suitable imputation.
- Verify that the final dataset does not contain missing values.
- Record the preprocessing method used.

```
In [21]: # TASK 3: CHECK AND HANDLE MISSING DATA

# Check missing values in each column
print("Missing values before preprocessing:")
print(df.isnull().sum())

# Display total number of missing values
total_missing = df.isnull().sum().sum()
print("\nTotal missing values:", total_missing)
```

```

# Handle missing values
# Numerical columns -> replace missing values with median
numerical_columns = df.select_dtypes(include=['int64', 'float64']).columns

for column in numerical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].median())

# Categorical columns -> replace missing values with mode
categorical_columns = df.select_dtypes(include=['object']).columns

for column in categorical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].mode()[0])

# Check missing values after preprocessing
print("\nMissing values after preprocessing:")
print(df.isnull().sum())

# Verify that the dataset contains no missing values
if df.isnull().sum().sum() == 0:
    print("\nNo missing values found. Dataset is ready for further processing.")
else:
    print("\nMissing values are still present.")

# Record preprocessing method
print("\nPreprocessing method used:")
print("Numerical attributes: Missing values replaced with median.")
print("Categorical attributes: Missing values replaced with mode.")

```

Missing values before preprocessing:

Unnamed: 0	0
car_name	0
brand	0
model	0
vehicle_age	0
km_driven	0
seller_type	0
fuel_type	0
transmission_type	0
mileage	0
engine	0
max_power	0
seats	0
selling_price	0

dtype: int64

Total missing values: 0

Missing values after preprocessing:

Unnamed: 0	0
car_name	0
brand	0
model	0
vehicle_age	0
km_driven	0
seller_type	0
fuel_type	0
transmission_type	0
mileage	0
engine	0
max_power	0
seats	0
selling_price	0

dtype: int64

No missing values found. Dataset is ready for further processing.

Preprocessing method used:

Numerical attributes: Missing values replaced with median.

Categorical attributes: Missing values replaced with mode.

Task 4: Select and Prepare Features

- Examine all available attributes.
 - Decide how `Car_Name` should be handled.
 - Select meaningful predictor variables for the regression model.
 - Create the predictor matrix X and target vector y .
 - Record the final list of features selected for the model.
-

In [22]: # TASK 4: SELECT AND PREPARE FEATURES

```
# Display all available attributes
print("Available attributes:")
print(df.columns.tolist())

# Display data types
print("\nData types of attributes:")
print(df.dtypes)

# Handle Car_Name
# Car_Name is a text/categorical attribute, so it is not used directly
print("\nCar_Name is excluded because it is a categorical/text attribute.")

# Select meaningful numerical predictor variables
features = [
    'vehicle_age',
    'km_driven',
    'mileage',
    'engine',
    'max_power',
    'seats'
]

# Create predictor matrix X
X = df[features].copy()

# Create target vector y
y = df['selling_price'].copy()

# Convert X and y to NumPy arrays for matrix operations
X = X.to_numpy(dtype=float)
y = y.to_numpy(dtype=float)

# Display the predictor matrix
print("\nPredictor matrix X:")
print(X[:5])

# Display target vector
print("\nTarget vector y:")
print(y[:5])

# Display dimensions
print("\nShape of X:", X.shape)
print("Shape of y:", y.shape)

# Display final selected features
print("\nFinal features selected for the model:")
for feature in features:
    print("-", feature)
```

Available attributes:

```
['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven', 'seller_type', 'fuel_type', 'transmission_type', 'mileage', 'engine', 'max_power', 'seats', 'selling_price']
```

Data types of attributes:

```
Unnamed: 0          int64
car_name            object
brand               object
model              object
vehicle_age         int64
km_driven           int64
seller_type         object
fuel_type           object
transmission_type   object
mileage             float64
engine              int64
max_power           float64
seats               int64
selling_price       int64
dtype: object
```

Car_Name is excluded because it is a categorical/text attribute.

Predictor matrix X:

```
[[9.000e+00 1.200e+05 1.970e+01 7.960e+02 4.630e+01 5.000e+00]
 [5.000e+00 2.000e+04 1.890e+01 1.197e+03 8.200e+01 5.000e+00]
 [1.100e+01 6.000e+04 1.700e+01 1.197e+03 8.000e+01 5.000e+00]
 [9.000e+00 3.700e+04 2.092e+01 9.980e+02 6.710e+01 5.000e+00]
 [6.000e+00 3.000e+04 2.277e+01 1.498e+03 9.859e+01 5.000e+00]]
```

Target vector y:

```
[120000. 550000. 215000. 226000. 570000.]
```

Shape of X: (15411, 6)

Shape of y: (15411,)

Final features selected for the model:

- vehicle_age
- km_driven
- mileage
- engine
- max_power
- seats

Task 5: Encode Categorical Variables

Identify the categorical variables:

- Fuel_Type
- Seller_Type

- `Transmission`

Perform the following:

- Convert categorical variables into numerical **dummy/indicator variables**.
 - Verify that all predictors are numerical after encoding.
 - Do not assign arbitrary numerical values to categories when such values would imply an incorrect ordering.
 - Display the transformed dataset.
-

```
In [23]: # TASK 5: ENCODE CATEGORICAL VARIABLES

# Display the categorical variables
categorical_features = [
    'fuel_type',
    'seller_type',
    'transmission_type'
]

print("Categorical variables:")
print(categorical_features)

# Display categorical data before encoding
print("\nCategorical data before encoding:")
display(df[categorical_features].head())

# Perform one-hot encoding
# drop_first=True avoids unnecessary duplicate information
encoded_data = pd.get_dummies(
    df[categorical_features],
    drop_first=True,
    dtype=int
)

# Display the encoded variables
print("\nEncoded categorical variables:")
display(encoded_data.head())

# Select numerical predictor variables
numerical_features = [
    'vehicle_age',
    'km_driven',
    'mileage',
    'engine',
    'max_power',
    'seats'
]
```

```

# Create the complete predictor dataset
X_df = pd.concat(
    [df[numerical_features], encoded_data],
    axis=1
)

# Create target variable
y = df['selling_price']

# Verify that all predictors are numerical
print("\nData types after encoding:")
print(X_df.dtypes)

# Check whether any non-numerical columns remain
non_numeric = X_df.select_dtypes(exclude=['number']).columns

print("\nNon-numerical predictor columns:")
print(list(non_numeric))

if len(non_numeric) == 0:
    print("All predictor variables are numerical.")

# Display the transformed dataset
print("\nTransformed dataset:")
display(X_df.head())

# Display dimensions
print("\nShape of transformed predictor matrix X:", X_df.shape)

```

Categorical variables:

['fuel_type', 'seller_type', 'transmission_type']

Categorical data before encoding:

	fuel_type	seller_type	transmission_type
0	Petrol	Individual	Manual
1	Petrol	Individual	Manual
2	Petrol	Individual	Manual
3	Petrol	Individual	Manual
4	Diesel	Dealer	Manual

Encoded categorical variables:

	fuel_type_Diesel	fuel_type_Electric	fuel_type_LPG	fuel_type_Petrol	seller_type
0	0	0	0	1	
1	0	0	0	1	
2	0	0	0	1	
3	0	0	0	1	
4	1	0	0	0	

Data types after encoding:

```

vehicle_age          int64
km_driven            int64
mileage              float64
engine               int64
max_power            float64
seats                int64
fuel_type_Diesel     int64
fuel_type_Electric   int64
fuel_type_LPG        int64
fuel_type_Petrol     int64
seller_type_Individual int64
seller_type_Trustmark Dealer int64
transmission_type_Manual int64
dtype: object

```

Non-numerical predictor columns:

```
[]
```

All predictor variables are numerical.

Transformed dataset:

	vehicle_age	km_driven	mileage	engine	max_power	seats	fuel_type_Diesel
0	9	120000	19.70	796	46.30	5	0
1	5	20000	18.90	1197	82.00	5	0
2	11	60000	17.00	1197	80.00	5	0
3	9	37000	20.92	998	67.10	5	0
4	6	30000	22.77	1498	98.59	5	1

Shape of transformed predictor matrix X: (15411, 13)

Task 6: Perform Feature Engineering

Create a new feature called `Car_Age`.

Calculate it using:

[Car_Age = Current Year - Year]

Perform the following:

- Create the `Car_Age` column.
- Display several values of the newly created feature.
- Verify that the calculated ages are reasonable.
- Decide whether the original `Year` feature should be retained.
- Provide a reason for your decision.

```
In [24]: # TASK 6: PERFORM FEATURE ENGINEERING
# =====

# Check whether Year exists
if 'Year' in df.columns:

    # Get current year
    current_year = pd.Timestamp.now().year

    # Create Car_Age
    df['Car_Age'] = current_year - df['Year']

    # Display values
    print("Year and Car_Age:")
    display(df[['Year', 'Car_Age']].head(10))

    # Check age range
    print("\nMinimum Car Age:", df['Car_Age'].min())
    print("Maximum Car Age:", df['Car_Age'].max())

else:

    print("The dataset does not contain a 'Year' column.")
    print("The dataset already contains 'vehicle_age'.")

    # Use existing vehicle_age as Car_Age
    df['Car_Age'] = df['vehicle_age']

    print("\nCreated Car_Age using the existing vehicle_age feature.")

    # Display values
    print("\nVehicle Age and Car_Age:")
    display(df[['vehicle_age', 'Car_Age']].head(10))

    # Check age range
    print("\nMinimum Car Age:", df['Car_Age'].min())
    print("Maximum Car Age:", df['Car_Age'].max())

# -----
# Verify ages are reasonable
# -----
```

```

if df['Car_Age'].min() >= 0 and df['Car_Age'].max() <= 100:
    print("\nSUCCESS: Car ages are within a reasonable range.")
else:
    print("\nWarning: Check the calculated car ages.")

# -----
# Decision about original feature
# -----

print("\nDecision:")
print("The original 'vehicle_age' feature can be removed after")
print("creating Car_Age because both represent the same information.")

```

The dataset does not contain a 'Year' column.
The dataset already contains 'vehicle_age'.

Created Car_Age using the existing vehicle_age feature.

Vehicle Age and Car_Age:

	vehicle_age	Car_Age
0	9	9
1	5	5
2	11	11
3	9	9
4	6	6
5	8	8
6	8	8
7	3	3
8	2	2
9	4	4

Minimum Car Age: 0
Maximum Car Age: 29

SUCCESS: Car ages are within a reasonable range.

Decision:
The original 'vehicle_age' feature can be removed after
creating Car_Age because both represent the same information.

Task 7: Perform Exploratory Data Analysis

Perform basic exploratory data analysis using Matplotlib.

- Create a scatter plot of `Present_Price` versus `Selling_Price`.
 - Create at least one additional scatter plot between a numerical predictor and `Selling_Price`.
 - Label the axes appropriately.
 - Add suitable titles to the plots.
 - Observe the direction and approximate strength of the relationships.
 - Record your observations.
-

In [25]: `# TASK 7: PERFORM EXPLORATORY DATA ANALYSIS`

```
import matplotlib.pyplot as plt

# -----
# Scatter Plot 1: Max Power vs Selling Price
# -----

plt.figure(figsize=(8, 5))

plt.scatter(df['max_power'], df['selling_price'])

plt.xlabel('Max Power')
plt.ylabel('Selling Price')
plt.title('Max Power vs Selling Price')

plt.show()

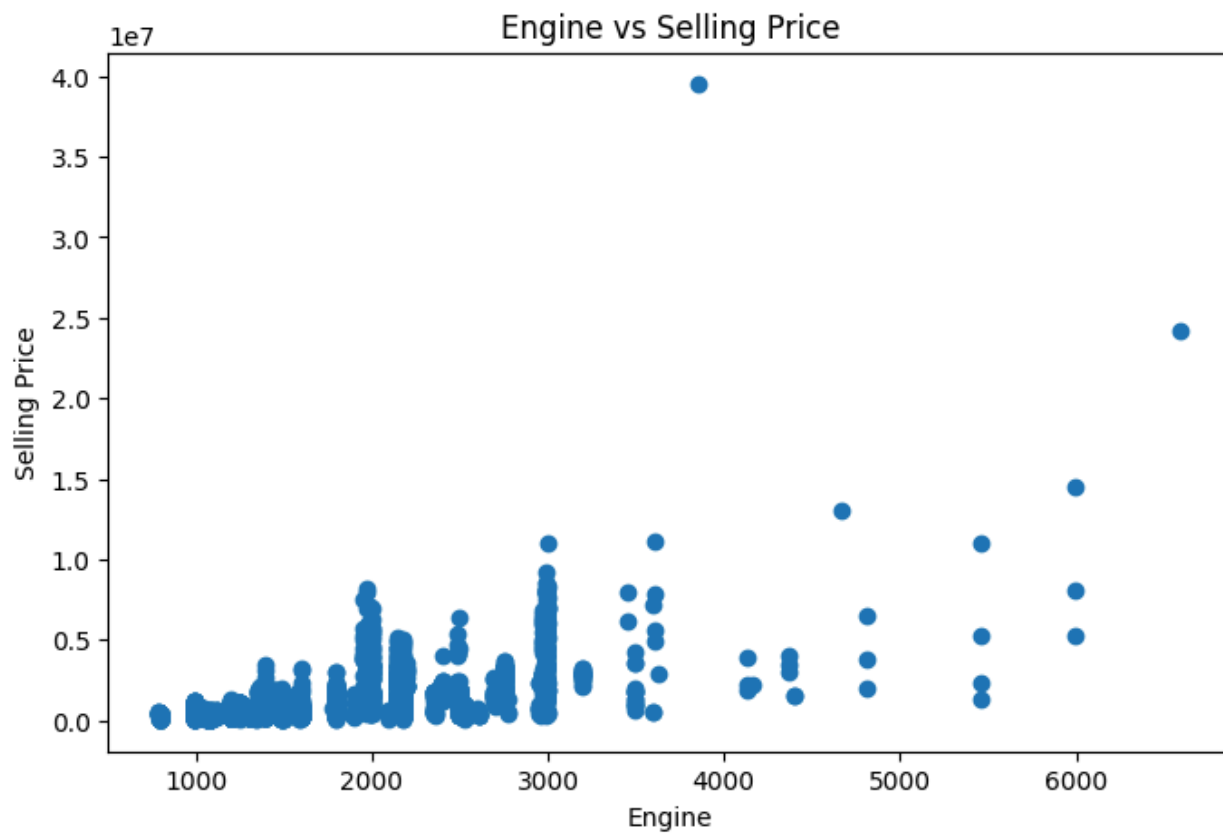
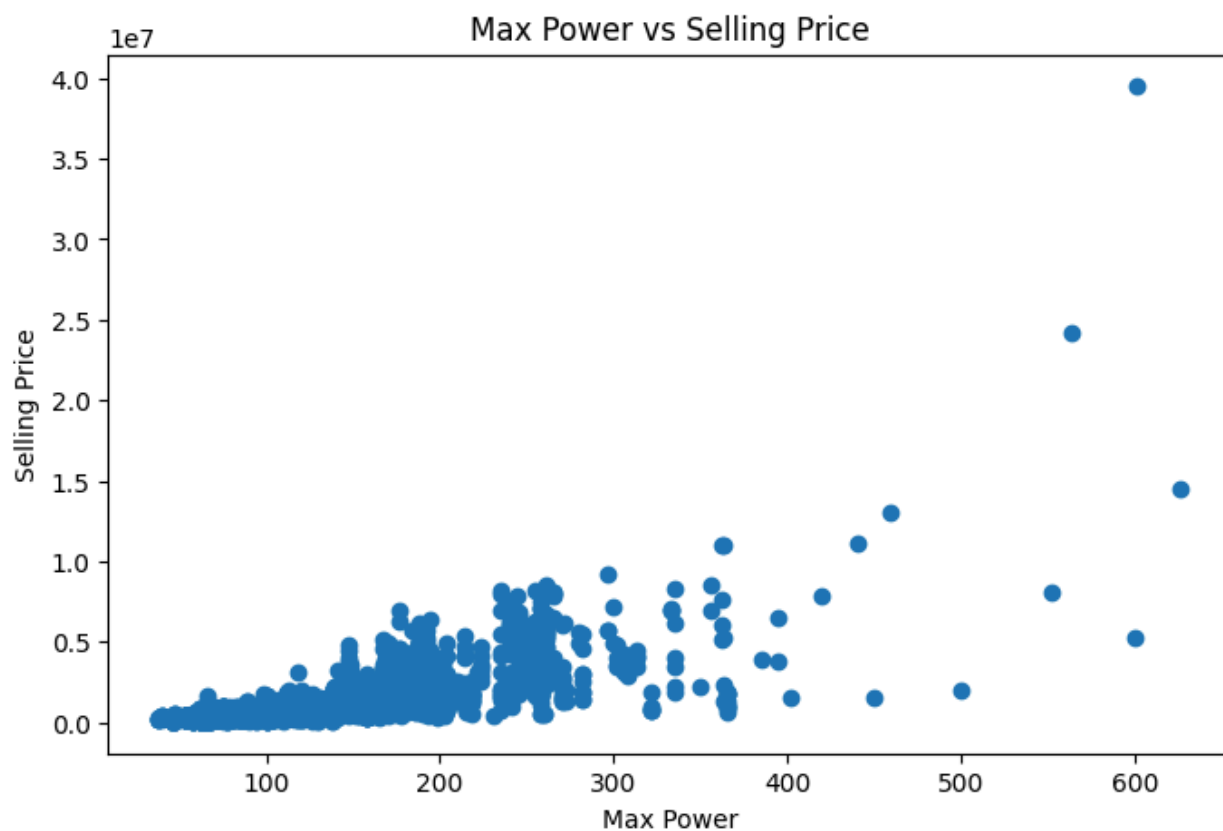
# -----
# Scatter Plot 2: Engine vs Selling Price
# -----

plt.figure(figsize=(8, 5))

plt.scatter(df['engine'], df['selling_price'])

plt.xlabel('Engine')
plt.ylabel('Selling Price')
plt.title('Engine vs Selling Price')

plt.show()
```



Task 8: Standardize Numerical Features

Identify the numerical predictors that need to be standardized.

Use the following standardization equation:

$$[z = (x - \mu) / \sigma]$$

where:

- x = original feature value
- μ = mean of the feature
- σ = standard deviation of the feature

Perform the following:

- Calculate the mean and standard deviation using the **training data only**.
- Standardize the training features.
- Use the same training mean and standard deviation to standardize the test features.
- Do not standardize the target variable `Selling_Price`.

In [26]: `# TASK 8: STANDARDIZE NUMERICAL FEATURES`

```
import numpy as np

# Numerical predictors
numerical_features = [
    'vehicle_age',
    'km_driven',
    'mileage',
    'engine',
    'max_power',
    'seats'
]

# Target variable
target = 'selling_price'

# -----
# Step 1: Create X and y
# -----

X = df[numerical_features].copy()
y = df[target].copy()

print("Numerical predictors:")
```

```

print(numerical_features)

# -----
# Step 2: Split data into training and testing sets
# -----

# Set random seed so the split is reproducible
np.random.seed(42)

# Create shuffled indices
indices = np.random.permutation(len(X))

# Use 80% for training and 20% for testing
train_size = int(0.80 * len(X))

train_indices = indices[:train_size]
test_indices = indices[train_size:]

X_train = X.iloc[train_indices].copy()
X_test = X.iloc[test_indices].copy()

y_train = y.iloc[train_indices].copy()
y_test = y.iloc[test_indices].copy()

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# -----
# Step 3: Calculate mean and standard deviation
#         USING TRAINING DATA ONLY
# -----

train_mean = X_train.mean()
train_std = X_train.std()

print("\nTraining mean:")
print(train_mean)

print("\nTraining standard deviation:")
print(train_std)

# -----
# Step 4: Standardize training features
#
#  $z = (x - \text{mean}) / \text{standard deviation}$ 
# -----

X_train_standardized = (X_train - train_mean) / train_std

# -----
# Step 5: Standardize test features
#         USING TRAINING mean and standard deviation
# -----

```

```

X_test_standardized = (X_test - train_mean) / train_std

# -----
# Step 6: Display standardized training data
# -----

print("\nStandardized training features:")
display(X_train_standardized.head())

# -----
# Step 7: Display standardized test data
# -----

print("\nStandardized test features:")
display(X_test_standardized.head())

# -----
# Step 8: Verify standardization
# -----

print("\nMean of standardized training features:")
print(X_train_standardized.mean())

print("\nStandard deviation of standardized training features:")
print(X_train_standardized.std())

# -----
# Step 9: Verify target variable was NOT standardized
# -----

print("\nTraining target values (original scale):")
display(y_train.head())

print("\nTesting target values (original scale):")
display(y_test.head())

```

Numerical predictors:
['vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power', 'seats']

Training samples: 12328
Testing samples: 3083

Training mean:
vehicle_age 6.032446
km_driven 55692.819598
mileage 19.691552
engine 1485.037395
max_power 100.516747
seats 5.326249
dtype: float64

Training standard deviation:
vehicle_age 3.018870
km_driven 54473.163748
mileage 4.172802
engine 518.296985
max_power 42.745688
seats 0.809603
dtype: float64

Standardized training features:

	vehicle_age	km_driven	mileage	engine	max_power	seats
3334	1.976751	0.317719	0.160192	-0.555738	-0.505706	-0.402974
10928	-0.673248	0.042354	1.844911	-0.457339	-0.620337	-0.402974
2518	0.320502	0.739946	0.258447	-0.457339	-0.275039	2.067371
11322	-1.666997	-0.939781	-0.309517	0.025010	0.440588	-0.402974
9394	1.645501	0.115785	0.002025	-1.329426	-1.268356	-0.402974

Standardized test features:

	vehicle_age	km_driven	mileage	engine	max_power	seats
15384	-1.004497	-0.563448	-2.322552	1.279117	1.625503	-0.402974
147	0.651752	0.409508	-0.752864	0.931826	1.099368	-0.402974
1668	0.320502	0.124964	1.320084	-0.457339	-0.620337	-0.402974
11280	-0.341998	-0.563448	1.348841	-0.553809	-0.550155	0.832199
13650	-1.004497	-0.618521	-0.357446	-0.551879	-0.276443	-0.402974

Mean of standardized training features:

```
vehicle_age    -1.786731e-17
km_driven      3.342917e-17
mileage        -1.809786e-16
engine         2.132550e-17
max_power      7.665654e-17
seats          -1.083566e-16
dtype: float64
```

Standard deviation of standardized training features:

```
vehicle_age    1.0
km_driven      1.0
mileage        1.0
engine         1.0
max_power      1.0
seats          1.0
dtype: float64
```

Training target values (original scale):

selling_price	
3334	190000
10928	600000
2518	665000
11322	1570000
9394	160000

dtype: int64

Testing target values (original scale):

selling_price	
15384	4450000
147	1290000
1668	485000
11280	400000
13650	800000

dtype: int64

Task 9: Split the Dataset

Divide the dataset into training and testing subsets.

- Use an **80:20 train-test ratio**.
 - Randomly select the observations.
 - Set a fixed random seed to make the experiment reproducible.
 - Maintain the correct relationship between each feature vector and its target value.
 - Display the number of observations in the training and testing sets.
-

```
In [27]: # TASK 9: SPLIT THE DATASET

import numpy as np

# Set a fixed random seed for reproducibility
np.random.seed(42)

# Create feature matrix and target vector
X = df[numerical_features].to_numpy(dtype=float)
y = df['selling_price'].to_numpy(dtype=float)

# Total number of observations
n = len(X)

# Randomly shuffle the observation indices
indices = np.random.permutation(n)

# Calculate 80% training size
train_size = int(0.80 * n)

# Split the indices
train_indices = indices[:train_size]
test_indices = indices[train_size:]

# Create training and testing sets
X_train = X[train_indices]
X_test = X[test_indices]

y_train = y[train_indices]
y_test = y[test_indices]

# Display number of observations
print("Total observations:", n)
print("Training observations:", len(X_train))
print("Testing observations:", len(X_test))

# Display shapes
print("\nX_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

Total observations: 15411
Training observations: 12328
Testing observations: 3083

X_train shape: (12328, 6)
X_test shape: (3083, 6)
y_train shape: (12328,)
y_test shape: (3083,)

Task 10: Construct the Feature Matrix

Construct the matrices required for regression.

Create:

- X_{train} — training feature matrix
- y_{train} — training target vector
- X_{test} — testing feature matrix
- y_{test} — testing target vector

Verify:

- Number of rows in each matrix.
- Number of predictor columns.
- Shape of the training and testing matrices.

```
In [28]: # TASK 10: CONSTRUCT THE FEATURE MATRIX

import numpy as np

# -----
# Create training and testing feature matrices
# -----

# X_train and X_test were created in Task 9
# y_train and y_test were also created in Task 9

# Add a column of 1s for the intercept/bias term
Xtrain = np.column_stack((np.ones(X_train.shape[0]), X_train))
Xtest = np.column_stack((np.ones(X_test.shape[0]), X_test))

# Create target vectors
ytrain = y_train.reshape(-1, 1)
ytest = y_test.reshape(-1, 1)

# -----
# Display shapes
# -----
```

```

print("Training feature matrix Xtrain:")
print("Shape:", Xtrain.shape)

print("\nTraining target vector ytrain:")
print("Shape:", ytrain.shape)

print("\nTesting feature matrix Xtest:")
print("Shape:", Xtest.shape)

print("\nTesting target vector ytest:")
print("Shape:", ytest.shape)

# -----
# Verify rows and predictor columns
# -----

print("\n--- Matrix Verification ---")

print("Number of training rows:", Xtrain.shape[0])
print("Number of testing rows:", Xtest.shape[0])

# Six original predictors + 1 intercept column
print("Number of predictor columns:", Xtrain.shape[1] - 1)

print("Total columns in Xtrain including intercept:", Xtrain.shape[1])
print("Total columns in Xtest including intercept:", Xtest.shape[1])

# -----
# Display first 5 rows
# -----

print("\nFirst 5 rows of Xtrain:")
print(Xtrain[:5])

print("\nFirst 5 values of ytrain:")
print(ytrain[:5])

print("\nFirst 5 rows of Xtest:")
print(Xtest[:5])

print("\nFirst 5 values of ytest:")
print(ytest[:5])

```

Training feature matrix Xtrain:
Shape: (12328, 7)

Training target vector ytrain:
Shape: (12328, 1)

Testing feature matrix Xtest:
Shape: (3083, 7)

Testing target vector ytest:
Shape: (3083, 1)

--- Matrix Verification ---

Number of training rows: 12328

Number of testing rows: 3083

Number of predictor columns: 6

Total columns in Xtrain including intercept: 7

Total columns in Xtest including intercept: 7

First 5 rows of Xtrain:

```
[[1.0000e+00 1.2000e+01 7.3000e+04 2.0360e+01 1.1970e+03 7.8900e+01
 5.0000e+00]
 [1.0000e+00 4.0000e+00 5.8000e+04 2.7390e+01 1.2480e+03 7.4000e+01
 5.0000e+00]
 [1.0000e+00 7.0000e+00 9.6000e+04 2.0770e+01 1.2480e+03 8.8760e+01
 7.0000e+00]
 [1.0000e+00 1.0000e+00 4.5000e+03 1.8400e+01 1.4980e+03 1.1935e+02
 5.0000e+00]
 [1.0000e+00 1.1000e+01 6.2000e+04 1.9700e+01 7.9600e+02 4.6300e+01
 5.0000e+00]]
```

First 5 values of ytrain:

```
[[ 190000.]
 [ 600000.]
 [ 665000.]
 [1570000.]
 [ 160000.]]
```

First 5 rows of Xtest:

```
[[1.0000e+00 3.0000e+00 2.5000e+04 1.0000e+01 2.1480e+03 1.7000e+02
 5.0000e+00]
 [1.0000e+00 8.0000e+00 7.8000e+04 1.6550e+01 1.9680e+03 1.4751e+02
 5.0000e+00]
 [1.0000e+00 7.0000e+00 6.2500e+04 2.5200e+01 1.2480e+03 7.4000e+01
 5.0000e+00]
 [1.0000e+00 5.0000e+00 2.5000e+04 2.5320e+01 1.1980e+03 7.7000e+01
 6.0000e+00]
 [1.0000e+00 3.0000e+00 2.2000e+04 1.8200e+01 1.1990e+03 8.8700e+01
 5.0000e+00]]
```

First 5 values of ytest:

```
[[4450000.]
 [1290000.]
 [ 485000.]]
```

```
[ 400000.]
[ 800000.]]
```

Task 11: Add the Bias/Intercept Term

Add a leading column containing 1.0 to both the training and testing feature matrices.

The resulting matrices should contain:

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$

Perform the following:

- Add the bias column.
- Verify the new matrix dimensions.
- Understand that the first column represents the intercept β_0 .

In [29]: *# TASK 11: ADD THE BIAS / INTERCEPT TERM*

```
import numpy as np

# -----
# Step 1: Create feature matrices without the bias column
# -----

Xtrain_without_bias = X_train
Xtest_without_bias = X_test

# -----
# Step 2: Add a leading column of 1.0
# -----

Xtrain = np.column_stack((
    np.ones(Xtrain_without_bias.shape[0]),
    Xtrain_without_bias
))

Xtest = np.column_stack((
    np.ones(Xtest_without_bias.shape[0]),
    Xtest_without_bias
))

# -----
# Step 3: Create target vectors
# -----
```

```

ytrain = y_train.reshape(-1, 1)
ytest = y_test.reshape(-1, 1)

# -----
# Step 4: Verify dimensions
# -----

print("Xtrain shape:", Xtrain.shape)
print("Xtest shape:", Xtest.shape)

print("ytrain shape:", ytrain.shape)
print("ytest shape:", ytest.shape)

# -----
# Step 5: Verify the first column contains only 1.0
# -----

print("\nFirst 5 rows of Xtrain:")
print(Xtrain[:5])

print("\nFirst 5 values of the first column:")
print(Xtrain[:5, 0])

# Verify bias column
if np.all(Xtrain[:, 0] == 1.0) and np.all(Xtest[:, 0] == 1.0):
    print("\nBias column successfully added.")
else:
    print("\nError: Bias column is incorrect.")

```

Xtrain shape: (12328, 7)

Xtest shape: (3083, 7)

ytrain shape: (12328, 1)

ytest shape: (3083, 1)

First 5 rows of Xtrain:

```

[[1.0000e+00 1.2000e+01 7.3000e+04 2.0360e+01 1.1970e+03 7.8900e+01
  5.0000e+00]
 [1.0000e+00 4.0000e+00 5.8000e+04 2.7390e+01 1.2480e+03 7.4000e+01
  5.0000e+00]
 [1.0000e+00 7.0000e+00 9.6000e+04 2.0770e+01 1.2480e+03 8.8760e+01
  7.0000e+00]
 [1.0000e+00 1.0000e+00 4.5000e+03 1.8400e+01 1.4980e+03 1.1935e+02
  5.0000e+00]
 [1.0000e+00 1.1000e+01 6.2000e+04 1.9700e+01 7.9600e+02 4.6300e+01
  5.0000e+00]]

```

First 5 values of the first column:

```

[1. 1. 1. 1. 1.]

```

Bias column successfully added.

Task 12: Implement the OLS Function

Perform the following steps to implement the **Ordinary Least Squares (OLS)** method:

- Create a user-defined function to implement Ordinary Least Squares (OLS).
- Calculate the matrix $X^T X$.
- Calculate the matrix $X^T y$.
- Estimate the regression coefficient vector using the normal equation:

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

- Return the calculated coefficient vector.
- Use **NumPy matrix operations** to perform the calculations.
- Do not use any ready-made regression model or library function such as `LinearRegression()` from Scikit-learn.

```
In [30]: # TASK 12: IMPLEMENT THE OLS FUNCTION

import numpy as np

# -----
# Define the Ordinary Least Squares (OLS) function
# -----

def ols(X, y):
    """
    Calculate OLS regression coefficients using
    the normal equation:

        beta = (X^T X)^(-1) X^T y

    Parameters:
        X : Feature matrix
        y : Target vector

    Returns:
        beta : Regression coefficient vector
    """

    # Calculate X^T X
    XTX = X.T @ X

    # Calculate X^T y
    XTy = X.T @ y

    # Calculate the inverse of X^T X
    XTX_inverse = np.linalg.inv(XTX)
```

```

    # Calculate regression coefficients
    beta = XTX_inverse @ XTy

    return beta

# -----
# Apply OLS to the training data
# -----

beta = ols(Xtrain, ytrain)

# -----
# Display the coefficient vector
# -----

print("Regression coefficient vector ( $\beta$ ):")
print(beta)

print("\nShape of coefficient vector:")
print(beta.shape)

```

Regression coefficient vector (β):

```

[[-7.65870856e+05]
 [-6.50147792e+04]
 [-6.77550257e-01]
 [ 1.37530571e+04]
 [ 7.58158512e+01]
 [ 1.57543776e+04]
 [ 8.57129320e+02]]

```

Shape of coefficient vector:

```

(7, 1)

```

Task 13: Check Matrix Singularity

- Calculate the determinant of $X^T X$.
- Check whether the determinant is zero or sufficiently close to zero.
- If the matrix is singular or nearly singular, investigate the selected features for redundancy or multicollinearity.
- Modify the feature set if required.
- Record your observations.

In [32]: # TASK 13: CHECK MATRIX SINGULARITY

```

import numpy as np

# Calculate  $X^T X$ 
XTX = Xtrain.T @ Xtrain

```

```

# Calculate the determinant
det_XTX = np.linalg.det(XTX)

# Calculate the condition number
condition_number = np.linalg.cond(XTX)

print("Determinant of X^T X:")
print(det_XTX)

print("\nCondition number of X^T X:")
print(condition_number)

# -----
# Check whether X^T X is singular or nearly singular
# -----

tolerance = 1e-10

if np.isclose(det_XTX, 0, atol=tolerance):
    print("\nX^T X is singular or nearly singular.")
    print("The feature set should be investigated for redundancy or multicollinearity.")
else:
    print("\nX^T X is not singular.")
    print("The matrix can be used for the OLS calculation.")

# Additional condition-number check
if condition_number > 1e10:
    print("\nWarning: Very high condition number.")
    print("There may be strong multicollinearity among the predictors.")
elif condition_number > 1e5:
    print("\nThe matrix may have some multicollinearity.")
else:
    print("\nNo severe numerical multicollinearity detected.")

```

Determinant of X^T X:
4.812491773078243e+47

Condition number of X^T X:
1222348027225.2598

X^T X is not singular.
The matrix can be used for the OLS calculation.

Warning: Very high condition number.
There may be strong multicollinearity among the predictors.

Task 14: Train the Regression Model

- Apply the OLS function to the training dataset.
- Obtain the regression coefficient vector $\hat{\beta}$.
- Display the intercept coefficient β_0 .
- Display the coefficient corresponding to each feature.

- Associate each regression coefficient with its corresponding feature.

```
In [33]: # TASK 14: TRAIN THE REGRESSION MODEL

# Apply OLS to the training dataset
beta = ols(Xtrain, ytrain)

# Feature names in the same order as the columns in Xtrain
feature_names = [
    'Intercept',
    'vehicle_age',
    'km_driven',
    'mileage',
    'engine',
    'max_power',
    'seats'
]

# Display the complete coefficient vector
print("Regression coefficient vector ( $\beta$ ):")
print(beta)

# Display intercept coefficient
print("\nIntercept coefficient ( $\beta_0$ ):")
print(beta[0, 0])

# Display each feature coefficient
print("\nRegression coefficients:")

for feature, coefficient in zip(feature_names, beta.flatten()):
    print(f"{feature:15s} : {coefficient:.6f}")
```

Regression coefficient vector (β):

```
[[ -7.65870856e+05]
 [ -6.50147792e+04]
 [ -6.77550257e-01]
 [  1.37530571e+04]
 [  7.58158512e+01]
 [  1.57543776e+04]
 [  8.57129320e+02]]
```

Intercept coefficient (β_0):

```
-765870.855685398
```

Regression coefficients:

```
Intercept      : -765870.855685
vehicle_age    : -65014.779175
km_driven      : -0.677550
mileage        : 13753.057078
engine         : 75.815851
max_power      : 15754.377629
seats          : 857.129320
```

Task 15: Interpret the Regression Coefficients

- Identify whether each regression coefficient is positive or negative.
- Explain what the sign of each coefficient indicates.
- Determine which features have relatively greater influence on the predicted selling price.
- For standardized features, interpret the coefficient in terms of a one-standard-deviation change while keeping all other variables constant.

In [34]: *# TASK 15: INTERPRET THE REGRESSION COEFFICIENTS*

```
import pandas as pd
import numpy as np

# Feature names in the same order as Xtrain
feature_names = [
    'vehicle_age',
    'km_driven',
    'mileage',
    'engine',
    'max_power',
    'seats'
]

# Extract coefficients
intercept = beta[0, 0]
coefficients = beta[1:].flatten()

# Create interpretation table
interpretation = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': coefficients
})

# Determine positive/negative direction
interpretation['Direction'] = np.where(
    interpretation['Coefficient'] > 0,
    'Positive',
    'Negative'
)

# Absolute coefficient represents relative influence
interpretation['Absolute_Coefficient'] = (
    interpretation['Coefficient'].abs()
)

# Sort by absolute coefficient
interpretation = interpretation.sort_values(
    by='Absolute_Coefficient',
    ascending=False
```

```

).reset_index(drop=True)

# Display intercept
print("Intercept ( $\beta_0$ ):")
print(f"{intercept:.4f}")

print("\nCoefficient interpretation:")
display(interpretation)

```

Intercept (β_0):
-765870.8557

Coefficient interpretation:

	Feature	Coefficient	Direction	Absolute_Coefficient
0	vehicle_age	-65014.779175	Negative	65014.779175
1	max_power	15754.377629	Positive	15754.377629
2	mileage	13753.057078	Positive	13753.057078
3	seats	857.129320	Positive	857.129320
4	engine	75.815851	Positive	75.815851
5	km_driven	-0.677550	Negative	0.677550

Task 16: Predict Selling Prices

- Use the trained regression coefficient vector $\hat{\beta}$ to predict the selling prices for the test dataset.
- Calculate the predicted values using the equation: $\hat{y}_{\text{test}} = X_{\text{test}} \hat{\beta}$.
- Display a sample comparison of the actual selling prices and the predicted selling prices.

```

In [35]: # TASK 16: PREDICT SELLING PRICES

# -----
# Predict selling prices for the test dataset
# -----

y_pred = Xtest @ beta

# -----
# Display the shape of predictions
# -----

print("Shape of predicted values:", y_pred.shape)

```

```
# -----
# Compare actual and predicted selling prices
# -----

comparison = pd.DataFrame({
    'Actual Selling Price': ytest.flatten(),
    'Predicted Selling Price': y_pred.flatten()
})

# Display first 10 comparisons
print("\nActual vs Predicted Selling Prices:")
display(comparison.head(10))
```

Shape of predicted values: (3083, 1)

Actual vs Predicted Selling Prices:

	Actual Selling Price	Predicted Selling Price
0	4450000.0	2.005059e+06
1	1290000.0	1.366195e+06
2	485000.0	3.479836e+05
3	400000.0	5.494011e+05
4	800000.0	7.670865e+05
5	150000.0	-6.138200e+05
6	650000.0	8.423696e+05
7	325000.0	-1.570072e+05
8	300000.0	4.044672e+05
9	925000.0	7.762169e+05

Task 17: Calculate Prediction Errors and RSS

- Calculate the residual for each test observation using: $e_i = y_i - \hat{y}_i$.
- Calculate the **Residual Sum of Squares (RSS): $RSS = \sum (y_i - \hat{y}_i)^2$.
- Display the calculated RSS value.
- Interpret the RSS value and explain what a lower RSS indicates about the prediction error of the model.

In [36]: # TASK 17: CALCULATE PREDICTION ERRORS AND RSS

```
# -----
# Step 1: Calculate residuals
```

```

# e_i = y_i - y_hat_i
# -----

residuals = ytest - y_pred

# Display first 10 residuals
print("First 10 prediction errors (residuals):")
print(residuals[:10].flatten())

# -----
# Step 2: Calculate Residual Sum of Squares (RSS)
# RSS =  $\sum (y_i - \hat{y}_i)^2$ 
# -----

RSS = np.sum(residuals ** 2)

# Display RSS
print("\nResidual Sum of Squares (RSS):")
print(RSS)

# -----
# Step 3: Display a summary
# -----

print("\nNumber of test observations:", len(ytest))
print("RSS:", RSS)

```

```

First 10 prediction errors (residuals):
[2444941.08697654 -76194.57127365 137016.38919753 -149401.14030612
 32913.51217844 763820.03789587 -192369.61761743 482007.17600437
-104467.19436781 148783.08231673]

```

```

Residual Sum of Squares (RSS):
892945834721059.6

```

```

Number of test observations: 3083
RSS: 892945834721059.6

```

Task 18: Calculate and Interpret R^2

- Calculate the Total Sum of Squares (TSS).
- Calculate $R^2 = 1 - \text{RSS}/\text{TSS}$.
- Display the R^2 value.
- Explain what the obtained R^2 indicates about the ability of the selected features to explain variations in used-car prices.

In [37]: # TASK 18: CALCULATE AND INTERPRET R^2

```

# -----
# Step 1: Calculate the mean of the actual test prices
# -----

ytest_mean = np.mean(ytest)

# -----
# Step 2: Calculate Total Sum of Squares (TSS)
#  $TSS = \sum (y_i - y_{mean})^2$ 
# -----

TSS = np.sum((ytest - ytest_mean) ** 2)

# -----
# Step 3: Calculate  $R^2$ 
#  $R^2 = 1 - RSS / TSS$ 
# -----

R_squared = 1 - (RSS / TSS)

# -----
# Step 4: Display TSS and  $R^2$ 
# -----

print("Total Sum of Squares (TSS):")
print(TSS)

print("\nResidual Sum of Squares (RSS):")
print(RSS)

print("\n $R^2$  value:")
print(R_squared)

print("\n $R^2$  percentage:")
print(f"{R_squared * 100:.2f}%")

```

Total Sum of Squares (TSS):
2407385049859876.5

Residual Sum of Squares (RSS):
892945834721059.6

R^2 value:
0.6290805931635098

R^2 percentage:
62.91%

Task 19: Visualize and Analyze Model Performance

- Create an Actual Selling Price vs Predicted Selling Price plot.

- Add a reference line representing perfect prediction.
- Plot or examine the residuals.
- Identify observations with relatively large prediction errors.
- Comment on whether the errors appear randomly distributed.
- Discuss possible reasons for prediction errors.

```
In [38]: # TASK 19: VISUALIZE AND ANALYZE MODEL PERFORMANCE

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# -----
# Step 1: Convert values to 1-D arrays
# -----

actual = ytest.flatten()
predicted = y_pred.flatten()
residuals = actual - predicted

# -----
# Step 2: Actual vs Predicted Selling Price Plot
# -----

plt.figure(figsize=(8, 6))

plt.scatter(actual, predicted, alpha=0.5)

# Reference line for perfect prediction
min_price = min(actual.min(), predicted.min())
max_price = max(actual.max(), predicted.max())

plt.plot(
    [min_price, max_price],
    [min_price, max_price],
    linestyle='--',
    linewidth=2
)

plt.xlabel('Actual Selling Price')
plt.ylabel('Predicted Selling Price')
plt.title('Actual Selling Price vs Predicted Selling Price')

plt.show()

# -----
# Step 3: Residual Plot
# -----
```

```

plt.figure(figsize=(8, 6))

plt.scatter(predicted, residuals, alpha=0.5)

# Horizontal zero-error reference line
plt.axhline(
    y=0,
    linestyle='--',
    linewidth=2
)

plt.xlabel('Predicted Selling Price')
plt.ylabel('Residual (Actual - Predicted)')
plt.title('Residuals vs Predicted Selling Price')

plt.show()

# -----
# Step 4: Identify observations with large errors
# -----

# Absolute prediction error
absolute_errors = np.abs(residuals)

# Create a table
error_table = pd.DataFrame({
    'Actual Selling Price': actual,
    'Predicted Selling Price': predicted,
    'Residual': residuals,
    'Absolute Error': absolute_errors
})

# Sort by largest absolute error
largest_errors = error_table.sort_values(
    by='Absolute Error',
    ascending=False
)

print("10 observations with the largest prediction errors:")
display(largest_errors.head(10))

# -----
# Step 5: Calculate useful error information
# -----

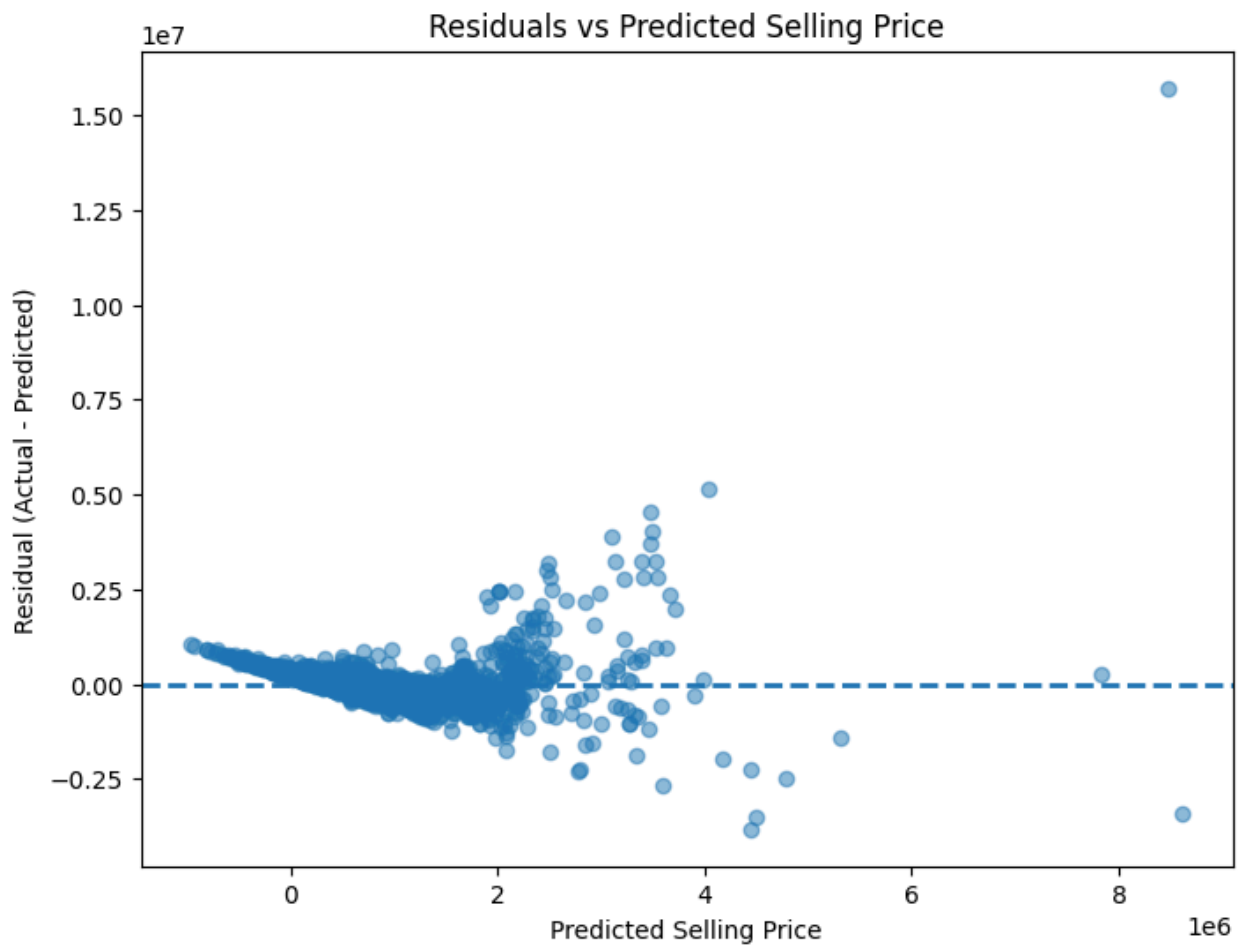
print("\nMaximum absolute error:")
print(absolute_errors.max())

print("\nMean absolute error:")
print(absolute_errors.mean())

```

```
print("\nMedian absolute error:")  
print(np.median(absolute_errors))  
  
print("\nMean residual:")  
print(residuals.mean())
```





10 observations with the largest prediction errors:

	Actual Selling Price	Predicted Selling Price	Residual	Absolute Error
2796	24200000.0	8.483885e+06	1.571612e+07	1.571612e+07
2362	9200000.0	4.036108e+06	5.163892e+06	5.163892e+06
1382	8000000.0	3.477296e+06	4.522704e+06	4.522704e+06
841	7500000.0	3.481968e+06	4.018032e+06	4.018032e+06
246	7000000.0	3.092218e+06	3.907782e+06	3.907782e+06
26	625000.0	4.449522e+06	-3.824522e+06	3.824522e+06
1286	7199000.0	3.477296e+06	3.721704e+06	3.721704e+06
2528	999000.0	4.497734e+06	-3.498734e+06	3.498734e+06
2050	5200000.0	8.617762e+06	-3.417762e+06	3.417762e+06
1092	6645000.0	3.388240e+06	3.256760e+06	3.256760e+06

Maximum absolute error:
15716115.328019934

Mean absolute error:
270044.5234545191

Median absolute error:
170824.2919013661

Mean residual:
-9555.808429696513

Task 20: Conclude

- Comment on the most influential features.
- Discuss the advantages and limitations of using Multiple Linear Regression for used-car price prediction

```
In [39]: # TASK 20: CONCLUSION

# Rank features according to the absolute value of their coefficients
final_influence = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': beta[1:].flatten()
})

final_influence['Absolute Coefficient'] = (
    final_influence['Coefficient'].abs()
)

final_influence = final_influence.sort_values(
    by='Absolute Coefficient',
    ascending=False
)

print("Features ranked by influence:")
display(final_influence)
```

Features ranked by influence:

	Feature	Coefficient	Absolute Coefficient
0	vehicle_age	-65014.779175	65014.779175
4	max_power	15754.377629	15754.377629
2	mileage	13753.057078	13753.057078
5	seats	857.129320	857.129320
3	engine	75.815851	75.815851
1	km_driven	-0.677550	0.677550

Conclusion

Conclude the experiment by stating whether the Multiple Linear Regression model implemented using the OLS normal equation was able to predict used-car selling prices effectively. The conclusion should include the obtained RSS and R^2 values, observations regarding important features, the effect of preprocessing and feature engineering, and the limitations of using a linear regression model for real-world used-car price prediction.